

DESARROLLO PARA DISPOSITIVOS INTELIGENTES

PRACTICA 2.5

Integracion API OpenWeatherMap
y Generacion de APK con Firma Digital

Unidad	Semanas	Duracion	Puntos
Unidad 2 - Flutter + BLE	7 - 8	5 horas	100 pts

Mayo – Agosto 2026

1. OBJETIVO

En esta practica conectaras la aplicacion Flutter con la API REST de OpenWeatherMap para mostrar datos climaticos reales, gestionaras la API key de forma segura usando variables de entorno, y generaras un APK firmado listo para instalar en un dispositivo Android.

1.1 Objetivos especificos

1. Obtener API key gratuita de OpenWeatherMap.
2. Gestionar credenciales con flutter_dotenv (nunca en codigo).
3. Implementar WeatherService con llamadas HTTP reales y manejo de errores.
4. Actualizar el modelo Weather con deserializacion JSON segura.
5. Integrar datos reales en la UI via WeatherProvider.
6. Crear keystore, firmar y generar APK release.
7. Instalar y probar el APK en dispositivo fisico.

1.2 Relacion con practicas anteriores

Practica	Lo que aporta a P2.5
P2.1 - Setup Flutter	Entorno configurado, proyecto base
P2.2 - UI 3 pantallas	Pantallas donde se mostraran datos reales
P2.3 - Logica y estado	WeatherProvider y Weather model (se actualizan aqui)
P2.4 - BLE	BLEService ya integrado; P2.5 agrega capa HTTP

2. PREPARACION DEL ENTORNO

2.1 Obtener API key de OpenWeatherMap

1. Abre <https://openweathermap.org/api> en tu navegador.
2. Haz clic en Subscribe bajo Current Weather Data (plan Free, 60 llamadas/minuto).
3. Crea una cuenta gratuita o inicia sesion.
4. Ve a tu perfil -> API keys. Copia la clave generada automaticamente; tiene este aspecto:

```
a1b2c3d4e5f67890abcdef1234567890
```

IMPORTANTE

La API key tarda entre 10 y 60 minutos en activarse tras el registro.
Si recibes error 401 al principio, espera y vuelve a intentarlo.

2.2 Agregar dependencias al proyecto

1. Abre pubspec.yaml en la raiz del proyecto y agrega las dependencias:

pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.1.2  
  http: ^1.2.1 # Llamadas HTTP  
  flutter_dotenv: ^5.1.0 # Lectura de .env  
  
flutter:  
  assets:  
    - .env # Incluir .env como asset
```

2. Instala las dependencias:

```
flutter pub get
```

2.3 Crear y proteger el archivo .env

1. Crea el archivo `.env` en la raíz del proyecto (al mismo nivel que `pubspec.yaml`):

```
.env
OPENWEATHER_API_KEY=tu_api_key_aqui
OPENWEATHER_BASE_URL=https://api.openweathermap.org/data/2.5/weather
```

2. Agrega `.env` al `.gitignore` para que NUNCA se suba al repositorio:

```
.gitignore
# Al final de .gitignore (raíz del proyecto)
.env

# Tambien agregar en android/.gitignore
*.jks
*.keystore
local.properties
```

3. Verifica que `.gitignore` funciona antes de hacer cualquier commit:

```
git status

# El archivo .env NO debe aparecer en la lista.
# Si aparece, detente y revisa el paso anterior antes de continuar.
```

CRITICO - PENALIZACION -50 PUNTOS

Si el archivo `.env` o la API key aparecen en cualquier commit del repositorio, la penalización es de -50 puntos y no tiene recuperación. Verifica siempre con `'git status'` antes de hacer commit.

3. CONFIGURACION Y CAPA DE SERVICIOS

3.1 AppConfig - Leer variables de entorno

Crea el archivo `lib/config/app_config.dart`. Esta clase centraliza el acceso a todas las variables de entorno:

<code>lib/config/app_config.dart</code>
<code>import 'package:flutter_dotenv/flutter_dotenv.dart';</code>
<code>class AppConfig {</code>
<code> // Getters que leen del .env en tiempo de ejecucion</code>
<code> static String get apiKey =></code>
<code> dotenv.env['OPENWEATHER_API_KEY'] ?? '';</code>
<code> static String get baseUrl =></code>
<code> dotenv.env['OPENWEATHER_BASE_URL'] ??</code>
<code> 'https://api.openweathermap.org/data/2.5/weather';</code>
<code> // Validar que las variables esten cargadas</code>
<code> static bool isConfigured() {</code>
<code> return apiKey.isNotEmpty && baseUrl.isNotEmpty;</code>
<code> }</code>
<code>}</code>

3.2 Weather Model - Deserializacion segura

Actualiza `lib/models/weather_model.dart` para mapear la respuesta real de la API. La API de OpenWeatherMap devuelve un JSON con esta estructura:

Respuesta JSON de OpenWeatherMap
<code>{</code>
<code> "name": "Queretaro",</code>
<code> "main": { "temp": 22.5, "humidity": 65 },</code>
<code> "weather": [{ "main": "Clear", "description": "cielo despejado" }],</code>
<code> "wind": { "speed": 3.2 }</code>
<code>}</code>

<code>lib/models/weather_model.dart</code>
<code>class Weather {</code>

```
final String city;
final int    temperature;
final String condition;
final String description;
final int    humidity;
final double windSpeed;

Weather({
    required this.city,
    required this.temperature,
    required this.condition,
    required this.description,
    required this.humidity,
    required this.windSpeed,
});

factory Weather.fromJson(Map<String, dynamic> json) {
    // Validar campos obligatorios antes de parsear
    if (!json.containsKey('main') || !json.containsKey('weather')) {
        throw const FormatException('Respuesta API incompleta');
    }
    if ((json['weather'] as List).isEmpty) {
        throw const FormatException('Sin datos de clima');
    }

    final temp = json['main']['temp'];
    if (temp is! num) {
        throw const FormatException('Temperatura invalida');
    }

    return Weather(
        city:      json['name'] ?? 'Desconocido',
        temperature: temp.toInt(),
        condition:  json['weather'][0]['main'] ?? 'Desconocido',
        description: json['weather'][0]['description'] ?? '',
        humidity:    (json['main']['humidity'] ?? 0) as int,
        windSpeed:   ((json['wind']['speed']) ?? 0).toDouble(),
    );
}

@override
String toString() =>
    'Weather($city: ${temperature}C, $condition, $humidity%, ${windSpeed}m/s)';
```

```
}
```

3.3 WeatherService - Llamadas HTTP

Crea o reemplaza lib/services/weather_service.dart:

lib/services/weather_service.dart
import 'dart:async';
import 'dart:io';
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/weather_model.dart';
import '../config/app_config.dart';
class WeatherService {
static const Duration _timeout = Duration(seconds: 10);
Future<Weather> getWeather(String city) async {
// 1. Validar entrada
if (city.trim().isEmpty) {
throw ArgumentError('La ciudad no puede estar vacia');
}
// 2. Sanitizar (solo letras, numeros y espacios)
final cleanCity = city.trim()
.replaceAll(RegExp(r'[^w\s]'), '');
// 3. Verificar que la API key esta configurada
if (!AppConfig.isConfigured()) {
throw Exception('API key no configurada. Revisa el archivo .env');
}
// 4. Construir URL (la API key va como parametro, NUNCA en el body)
final uri = Uri.parse(
'\${AppConfig.baseUrl}'
'?q=\$cleanCity'
'&appid=\${AppConfig.apiKey}'
'&units=metric'
'&lang=es',
);
try {

```
// 5. Ejecutar GET con timeout
final response = await http.get(uri).timeout(_timeout);

// 6. Manejar codigos de respuesta
switch (response.statusCode) {
  case 200:
    final json = jsonDecode(response.body) as Map<String, dynamic>;
    return Weather.fromJson(json);
  case 401:
    throw Exception('API key invalida o no activada aun');
  case 404:
    throw Exception('Ciudad "$city" no encontrada');
  case 429:
    throw Exception('Limite de llamadas excedido. Espera un momento');
  default:
    throw Exception('Error del servidor: ${response.statusCode}');
}
} on SocketException {
  throw Exception('Sin conexion a internet');
} on TimeoutException {
  throw Exception('Tiempo de espera agotado. Intenta de nuevo');
} on FormatException catch (e) {
  throw Exception('Respuesta inesperada de la API: $e');
}
}

// Consultar varias ciudades en paralelo
Future<List<Weather>> getWeatherForCities(List<String> cities) async {
  final futures = cities.map((c) => getWeather(c));
  return Future.wait(futures);
}
}
```


4. PROVIDER Y PANTALLA PRINCIPAL

4.1 WeatherProvider actualizado

Actualiza lib/providers/weather_provider.dart para manejar estados de carga y error:

lib/providers/weather_provider.dart
import 'package:flutter/material.dart';
import '../models/weather_model.dart';
import '../services/weather_service.dart';
class WeatherProvider extends ChangeNotifier {
final WeatherService _service = WeatherService();
Weather? _weather;
bool _isLoading = false;
String? _error;
Weather? get weather => _weather;
bool get isLoading => _isLoading;
String? get error => _error;
Future<void> fetchWeather(String city) async {
_isLoading = true;
_error = null;
notifyListeners();
try {
_weather = await _service.getWeather(city);
} catch (e) {
_error = e.toString().replaceFirst('Exception: ', '');
} finally {
_isLoading = false;
notifyListeners();
}
}
}

4.2 Cargar .env al iniciar la app

Actualiza lib/main.dart para cargar el archivo .env antes de arrancar la UI:

lib/main.dart
import 'package:flutter/material.dart';
import 'package:flutter_dotenv/flutter_dotenv.dart';
import 'package:provider/provider.dart';
import 'providers/weather_provider.dart';
import 'providers/ble_provider.dart';
import 'screens/home_screen.dart';
Future<void> main() async {
// Cargar variables de entorno ANTES de runApp
await dotenv.load(fileName: '.env');
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({super.key});
@override
Widget build(BuildContext context) {
return MultiProvider(
providers: [
ChangeNotifierProvider(create: (_) => WeatherProvider()),
ChangeNotifierProvider(create: (_) => BLEProvider()),
],
child: MaterialApp(
title: 'Climate App',
debugShowCheckedModeBanner: false,
theme: ThemeData(
colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
useMaterial3: true,
),
home: const HomeScreen(),
),
);
}
}

4.3 HomeScreen con datos reales

Actualiza lib/screens/home_screen.dart:

lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/weather_provider.dart';
class HomeScreen extends StatefulWidget {
const HomeScreen({super.key});
@override
State<HomeScreen> createState() => _HomeScreenState();
}
class _HomeScreenState extends State<HomeScreen> {
final _controller = TextEditingController();
@override
void initState() {
super.initState();
// Cargar ciudad por defecto al abrir
Future.microtask(() =>
context.read<WeatherProvider>().fetchWeather('Queretaro'),
);
}
@override
void dispose() {
_controller.dispose();
super.dispose();
}
void _search() {
final city = _controller.text.trim();
if (city.isNotEmpty) {
context.read<WeatherProvider>().fetchWeather(city);
FocusScope.of(context).unfocus();
}
}
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(
title: const Text('Climate App'),
centerTitle: true,

```
    ),  
    body: Column(  
      children: [  
        // Barra de busqueda  
        Padding(  
          padding: const EdgeInsets.all(16),  
          child: Row(  
            children: [  
              Expanded(  
                child: TextField(  
                  controller: _controller,  
                  decoration: const InputDecoration(  
                    hintText: 'Buscar ciudad...',  
                    border: OutlineInputBorder(),  
                    prefixIcon: Icon(Icons.search),  
                  ),  
                  onSubmitted: (_) => _search(),  
                ),  
              ),  
              const SizedBox(width: 8),  
              ElevatedButton(  
                onPressed: _search,  
                child: const Text('Buscar'),  
              ),  
            ],  
          ),  
        ),  
        // Contenido dinamico  
        Expanded(  
          child: Consumer<WeatherProvider>(  
            builder: (context, wp, _) {  
              if (wp.isLoading) {  
                return const Center(child: CircularProgressIndicator());  
              }  
              if (wp.error != null) {  
                return Center(  
                  child: Column(  
                    mainAxisAlignment: MainAxisAlignment.center,  
                    children: [  
                      const Icon(Icons.cloud_off, size: 64, color: Colors.red),  
                      const SizedBox(height: 16),  
                      Text(wp.error!, textAlign: TextAlign.center),  
                      const SizedBox(height: 24),  
                      ElevatedButton(  

```

```

        onPressed: () => wp.fetchWeather('Queretaro'),
        child: const Text('Reintentar'),
      ),
    ],
  ),
);
}
if (wp.weather == null) {
  return const Center(child: Text('Ingresa una ciudad'));
}
final w = wp.weather!;
return SingleChildScrollView(
  padding: const EdgeInsets.all(24),
  child: Column(
    children: [
      Text(w.city,
        style: Theme.of(context).textTheme.headlineMedium,
        const SizedBox(height: 8),
      Text('${w.temperature}C',
        style: const TextStyle(
          fontSize: 72, fontWeight: FontWeight.bold,
          color: Colors.blue)),
      Text(w.description,
        style: const TextStyle(fontSize: 20)),
      const SizedBox(height: 32),
      Row(
        mainAxisAlignment: MainAxisAlignment.spaceAround,
        children: [
          _stat('Humedad', '${w.humidity}%'),
          _stat('Viento', '${w.windSpeed} m/s'),
        ],
      ),
    ],
  ),
);
},
),
),
],
),
);
}

```

```
Widget _stat(String label, String value) => Column(
```

```
children: [  
  Text(label, style: const TextStyle(color: Colors.grey)),  
  const SizedBox(height: 4),  
  Text(value, style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),  
],  
);  
}
```

Ejecuta la app y busca tu ciudad para verificar que llegan datos reales:

```
flutter run
```

5. GENERACION DE APK CON FIRMA DIGITAL

5.1 Crear el Keystore

El keystore es el archivo que contiene las claves criptograficas con las que se firma el APK. Sin firma, Android no instala la app.

Ejecuta en la carpeta android/app de tu proyecto:

Crear Keystore
Windows (PowerShell)
cd android\app
keytool -genkey -v `
-keystore climate_app.jks `
-keyalg RSA -keysize 2048 -validity 10000 `
-alias climate_key
macOS / Linux / WSL2
cd android/app
keytool -genkey -v \
-keystore climate_app.jks \
-keyalg RSA -keysize 2048 -validity 10000 \
-alias climate_key

El comando pedira los datos del certificado. Completa asi:

Campo	Valor de ejemplo
Contrasena keystore	MiContrasenaSegura2025!
Contrasena clave	MiContrasenaSegura2025! (igual)
Nombre completo	Tu Nombre
Unidad organizacional	Ingenieria en Sistemas
Organizacion	Universidad
Ciudad	Queretaro
Estado	Queretaro
Codigo de pais	MX
Confirmacion final	yes

GUARDA LA CONTRASENA

Anota la contraseña del keystore en un lugar seguro (gestor de contraseñas).
 Si la pierdes no podrás actualizar la app en el futuro.
 El archivo .jks tampoco debe subirse a Git (ya lo excluiste en el .gitignore).

5.2 Configurar la firma en Gradle

Abre android/app/build.gradle y agrega la configuración de firma dentro del bloque android {}:

android/app/build.gradle	
android {	
// ... configuracion existente ...	
signingConfigs {	
release {	
keyAlias 'climate_key'	
keyPassword System.getenv('KEY_PASSWORD') ?: 'MiContrasenaSegura2025!'	
storeFile file('climate_app.jks')	
storePassword System.getenv('STORE_PASSWORD') ?: 'MiContrasenaSegura2025!'	
}	
}	
buildTypes {	
release {	
signingConfig signingConfigs.release	
minifyEnabled true	
shrinkResources true	
proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),	
'proguard-rules.pro'	
}	
}	
}	

NOTA DE SEGURIDAD

En esta práctica las contraseñas van directamente para simplicidad.
 En un proyecto real usarías variables de entorno (System.getenv) o el archivo local.properties (excluido del .gitignore).

5.3 Limpiar y generar el APK

```
# Desde la raíz del proyecto
```



```
flutter clean
flutter pub get
flutter build apk --release
```

Espera entre 2 y 4 minutos. Al terminar veras:

```
Built build/app/outputs/flutter-apk/app-release.apk (XX.X MB) .
```

5.4 Instalar en dispositivo fisico

Opcion A - Con adb (recomendado):

```
# Verificar que el dispositivo esta conectado
adb devices

# Instalar el APK
adb install build/app/outputs/flutter-apk/app-release.apk

# Resultado esperado:
# Performing Streamed Install
# Success
```

Opcion B - Manual:

- Copia el APK al dispositivo (USB o Google Drive).
- En el dispositivo, abre el administrador de archivos y localiza el APK.
- Toca el archivo -> Instalar.
- Si aparece advertencia de fuentes desconocidas, acepta solo para esta instalacion.

6. LISTA DE VERIFICACION

Fase 1 - API OpenWeatherMap

- ☐ API key obtenida en openweathermap.org
- ☐ Archivo .env creado con API key y URL base
- ☐ .env incluido en .gitignore (verificado con git status)
- ☐ flutter_dotenv y http agregados a pubspec.yaml
- ☐ app_config.dart creado y funcional
- ☐ weather_model.dart actualizado con fromJson seguro
- ☐ weather_service.dart con manejo de todos los errores HTTP
- ☐ weather_provider.dart actualizado
- ☐ main.dart carga .env antes de runApp
- ☐ App muestra temperatura real de al menos una ciudad
- ☐ App muestra mensaje de error si no hay conexion
- ☐ App muestra mensaje de error si la ciudad no existe

Fase 2 - Firma y APK

- ☐ climate_app.jks creado en android/app/
- ☐ Contraseña del keystore guardada en lugar seguro
- ☐ *.jks excluido del .gitignore
- ☐ build.gradle configurado con signingConfigs
- ☐ flutter build apk --release ejecutado sin errores
- ☐ APK instalado en dispositivo físico o emulador
- ☐ App se abre desde el icono del launcher (no desde IDE)
- ☐ Datos reales visibles en la app instalada

Entregables finales

- ☐ Commit: 'P2.5: API OpenWeatherMap + APK release'
- ☐ Screenshot de la app mostrando datos reales (para el reporte)
- ☐ APK subido como release en GitHub (v1.0-alpha)
- ☐ Reporte: configuración, pruebas y evidencias

7. RUBRICA DE EVALUACION

Criterio	Descripcion	Puntos
Integracion API	WeatherService conecta a OpenWeatherMap y muestra datos reales	30
Seguridad credenciales	API key en .env; nunca en codigo ni en commits	15
Manejo de errores	App no crashea; muestra mensajes para errores de red, timeout, ciudad invalida	15
Keystore y firma APK	climate_app.jks creado; build.gradle configurado correctamente	15
APK ejecutable	flutter build apk --release exitoso; APK instalado y funcional	20
Reporte y evidencias	Screenshot de app, commit limpio, APK como GitHub release	5

TOTAL: 100 puntos

Penalizaciones

Penalizacion	Puntos
API key en cualquier commit del repositorio	-50 (irrecuperable)
Keystore (.jks) en el repositorio	-20
APK no generado o no funcional	-20
Sin archivo .env (API key hardcodeada)	-15
Sin manejo de errores de red	-10
Sin reporte ni evidencias	-5

8. SOLUCION DE PROBLEMAS COMUNES

Problema	Causa probable	Solucion
Error 401 de la API	API key no activada aun	Espera 10-60 min y reintenta
Error 404	Ciudad escrita incorrectamente	Usa nombre en ingles: 'Queretaro' no 'Querétaro'
Sin conexion	Emulador sin internet	Settings -> Extended controls -> Cellular: activar
DotEnv no carga	.env no incluido en flutter.assets	Verifica que assets: [.env] este en pubspec.yaml
APK no instala	Modo desarrollador o USB debugging off	Activar Opciones de desarrollador en el dispositivo
Keystore error en build	Ruta o contraseña incorrecta en gradle	Verifica que storeFile apunta al .jks correcto
App crashea al abrir	dotenv.load no se llama antes de runApp	Asegurar que main() es async y llama await dotenv.load

9. SIGUIENTE PASO

Preparacion para la Evaluacion 1 (Parcial 1)

Con P2.5 completa, tienes todo lo necesario para la demostracion.

Lo que se evaluara en la demo (10 minutos):

- App corriendo en dispositivo fisico (no en emulador).
- Buscar una ciudad y mostrar datos reales de la API.
- Demostrar que la API key NO esta en el codigo ni en Git.
- Conectar con wearable via BLE y enviar temperatura (P2.4).
- Explicar la arquitectura: Provider, WeatherService, AppConfig.

Revisa la Lista de Cotejo Evaluacion 1 antes de presentar.

Resumen de practicas completadas

Practica	Contenido	Estado
P2.1	Setup Flutter + Android Studio	Completada
P2.2	UI: 3 pantallas navegables	Completada
P2.3	Logica Dart + state management	Completada

Practica	Contenido	Estado
P2.4	BLE movil <-> wearable	Completada
P2.5	API OpenWeatherMap + APK firmado	Esta practica